

```
if(len(right)>0):
    for i in right:
        newcabinet.append(i)
return(newcabinet)

left = [1,3,4,4,5,7,8,9]
right = [2,4,6,7,8,8,10,12,13,14]

newcab=merging(left,right)
```

[Listing 4-4](#): *An algorithm to merge two sorted lists*

The code in [Listing 4-4](#) creates `newcab`, a single list that contains all elements of `left` and `right`, merged and in order. You can run `print(newcab)` to see that our merging function worked.

From Merging to Sorting

Once we know how to merge, merge sort is within our grasp. Let's start by creating a simple merge sort function that works only on lists that have two or fewer elements. A one-element list is already sorted, so if we pass that as the input to our merge sort function, we should just return it unaltered. If we pass a two-element list to our merge sort function, we can split that list into two one-element lists (that are therefore already sorted) and call our merging function on those one-element lists to get a final, sorted two-element list. The following Python function accomplishes what we need:

```
import math

def mergesort_two_elements(cabinet):
    newcabinet = []
    if(len(cabinet) == 1):
        newcabinet = cabinet
    else:
        left = cabinet[:math.floor(len(cabinet)/2)]
        right = cabinet[math.floor(len(cabinet)/2):]
        newcabinet = merging(left,right)
    return(newcabinet)
```

This code relies on Python's list indexing syntax to split whatever `cabinet` we want to sort into a left cabinet and a right cabinet. You can see in the lines that define `left` and `right` that we're using `:math.floor(len(cabinet)/2)` and